

AI-Generated Versus Human Text: Introducing a New Dataset for Benchmarking and Analysis

Ali Al Bataineh , Rachel Sickler, Kerry Kurcz , and Kristen Pedersen 

Abstract—Artificial intelligence (AI) is increasingly embedded in our everyday lives. With the introduction of ChatGPT in November 2022 by OpenAI, people can now ask a bot to generate comprehensive writeups in seconds. This new transformative technology also introduces ethical, safety, and other general concerns. It is important to harness the power of AI to understand whether a body of text is generated by AI or whether it is organically human. In this article, we create and curate a medium-sized dataset of 10 000 records containing both human and machine-generated text and utilize it to train a reliable model to accurately distinguish between the two. First, we use DistilGPT-2 with various inputs to generate machine text. Then, we acquire an equal sample size of human-generated text. All the text is cleaned, explored, and visualized using the uniform manifold approximation and projection (UMAP) dimensionality reduction technique. Finally, the text is transformed into vectors using several techniques, including bag of words, term frequency-inverse document frequency, bidirectional encoder representations from transformer, and neural network-based embeddings. Machine learning experiments are then performed with traditional models such as logistic regression, random forest, and XGBoost, as well as deep learning models such as long short-term memory (LSTM), convolutional neural network (CNN), and CNN-LSTM. Across all vectorization strategies and machine learning algorithms, we measure accuracy, precision, recall, and F1 scores. We also time each exercise. Each model completes its training within an hour, and we observe scores above 90%. We then use the Shapley additive explanations (SHAP) package on machine learning models to explore if and how we can explain the model to further validate results. Lastly, we deploy our TF-IDF Random Forest model to a user-friendly web application using the Streamlit framework, allowing users without coding expertise to interact with the model.

Impact Statement—Due to the rapid proliferation of AI-generated content, which poses significant challenges in distinguishing it from human-authored text, our article provides

the research community with a balanced dataset of 10 000 records that include both AI-generated and human-generated text. The dataset offers a standardized benchmark for training and evaluating AI models that detect AI-generated content and contributes to the ongoing discussions in AI ethics and the field of natural language processing.

Index Terms—Artificial intelligence (AI), AI text detection, bidirectional encoder representations from transformer (BERT), ChatGPT, DistilGPT-2, explainable AI (XAI), OpenAI, SHAP package.

I. INTRODUCTION

A. Importance and Approaches to AI-Generated Text Detection

AI-GENERATED text detection is an important and difficult task in the fast-growing landscape of large language models (LLMs). With the increased proficiency of text generative AI, such as ChatGPT, many people cannot tell the difference between text written by a human or generated by AI. The approaches to detecting AI-generated text range from simple statistical methods to using generative models themselves for the task. The detection task is usually framed as binary classification. These methods include feature-based models such as support vector machines (SVM), random forest (RF), and neural networks (NN), process input vectors with text-based features including term frequency-inverse document frequency (TF-IDF), parts of speech, lemmas, punctuation count, and text length [1]. Statistical approaches like those using Markov methods can be also used as well as deep learning-based methods that leverage architectures such as recurrent neural networks (RNN), long short-term memory (LSTM), and gated recurrent units (GRU) for text analysis.

Generative adversarial networks (GAN) have been adapted to generate context for these models, and Reinforcement learning (RL) has been used to help fine-tune the outputs of neural network models [1]. While the trained models have varying degrees of accuracy, an article by Islam et al. demonstrated an extremely randomized trees model that outperformed other models with a 0.77 accuracy score when attempting to detect text that was generated by ChatGPT [2]. Still, a detection model created by Mitrović et al. used DistilBERT (a transformer model) to do the same ChatGPT detection task with a 0.98 accuracy score. This article also found that even these seemingly proficient models don't perform well when the text is rephrased using AI, with the detection accuracy falling to 0.79 [3]. Still, these models (and all models) only have the context they're

Received 1 February 2024; revised 3 August 2024; accepted 16 February 2025. Date of publication 20 February 2025; date of current version 31 July 2025. This work was supported by Norwich University Applied Research Institute and the AI Center at Norwich University, built under the grant "Advanced Computing through Experiential Education" from the Department of Education under Grant P116Z220106. The work of Ali Al Bataineh was supported by the National Science Foundation (NSF), the National Institutes of Health (NIH), the Department of Education, and other organizations. This article was recommended for publication by Associate Editor Annabel Latham upon evaluation of the reviewers' comments. (Corresponding author: Ali Al Bataineh.)

Ali Al Bataineh is with the Artificial Intelligence Center, Norwich University, Northfield, VT 05663 USA (e-mail: aalbatai@norwich.edu).

Rachel Sickler, Kerry Kurcz, and Kristen Pedersen are with Norwich University Applied Research Institutes, Northfield, VT 05663 USA.

Digital Object Identifier 10.1109/TAI.2025.3544183

provided and tend to only perform at even a moderately high level of accuracy on the output of the LLMs they were trained to detect. Using the generative models themselves seems like a promising way to detect the outputs of LLMs. Using a zero-shot method, the Grover model was used to demonstrate detection with 0.92 accuracy when limited to the domain on which the model was trained [4]. However, TF-IDF methods still outperform this method in general. State-of-the-art detection models are created by fine-tuning neural models with even a small sample of the data we seek to identify. This was demonstrated using RoBERTa-base (123M parameters) and RoBERTa-large (354M parameters) using the machine-generated text created with GPT-2 using multiple sampling methods to increase generalizability [1]. This suggests that there is a sampling threshold that would allow a detection method to perform general AI-generated text detection. Not every detection method employs machine learning. For example, text readability and comprehensibility scores such as the Gunning-Fog Index and the Flesch Index show promise and have predictive applications [1]. Vector databases are also used to assess similarity between prompt and response [5]. These provide a way for a human evaluator to make a determination as opposed to a strict binary classification problem resolved by a machine alone.

B. Challenges, Public Tools, and Future Directions

Some of the methods of detection have been deployed to publicly available web applications to help consumers distinguish between text generated by AI and that with a human author. None are 100% accurate and all have their limitations. GPTZero, which is trained specifically for ChatGPT, uses perplexity, a measure of text randomness, and burstiness, a measure of text complexity, as its methods of detection. Another tool, OpenAI Text Classifier, was created by OpenAI and claims to differentiate between human and AI-generated text and is trained on AI text generators including (but not limited to) ChatGPT. It has poor performance on short texts (less than 1000 characters), text generated from fine-tuned generative models, long text especially written by humans (identifies as written by AI), and text that is not in English. Some tools are very limited in scope, such as Writer.com's AI Content Detector which only allows a maximum of 1500 input characters. Copyleaks AI Content Detector touts a 99.12% detection accuracy rate and claims to work well with GPT-J, GPT-3, GPT-3.5, ChatGPT, GPT-4, and other related AI language models as well as multiple languages. Finally, the giant language model test room (GLTR) compares the prediction it would make, by word, to the next actual word in a submitted text and allows the user to decide if the text in its entirety should be classified as AI-generated. When these tools were tested for accuracy with ChatGPT responses to seven prompts in an article published in the *Journal of Applied Learning and Teaching* [6], GLTR classified all sample texts as human. OpenAI Text Classifier and Writer.com's AI Content Detector classified only 2 of the AI-generated texts accurately while GPTZero classified three correctly. Copyleaks performed the best, classifying five of the seven texts as generated by AI.

Even these tools which claim to have the highest accuracy on ChatGPT perform too inaccurately to be relied upon.

Unfortunately, the reliability of existing detection models plagues the AI-generated text detection field. The use of pseudorandom numbers in LLMs creates a distribution unlike the distribution of any text a human would create, and having even one line of AI-generated text within a human-authored text could throw a false positive [5]. In an article by Liang et al., they find that over half of the 91 Test of English as a Foreign Language (TOEFL) essays they studied were identified as AI-generated by seven different AI detection systems, while the essays written by native English speakers were correctly classified [7]. This could lead to further marginalizing groups where authors may exhibit a more limited linguistic variability. Perhaps detection tools should not always be used as binary classification, but as feedback for the user to adjust or improve writing style because what these models are detecting is writing that is derivative of the training data.

There are ways to mitigate some of the issues with AI detection and appropriate use, for instance: rigorous testing for reliability and bias as well as tactics like using paraphrasers after using AI to generate text to evade detection. Many deep learning models and embedding methods, by their nature, do not offer explainability, which could be important to establishing authentic text and understanding the limits of classifiers that could be restricting discourse online, in academia, and beyond. Having a diverse training data set from many different generators (human and AI) could mitigate some of the distribution issues, making detection systems more reliable, and simpler models could offer cleaner interpretability.

The training data used by researchers for creating and stress testing AI detection tools is as diverse as the models themselves. Some use publicly available datasets such as XSum [5] and the TweepFake Twitter dataset [1], while many also generated their own novel datasets manually, using a curated list of prompts to harvest AI-generated text from ChatGPT and Google's bidirectional and auto-regressive transformers (BARD) [3]. The human-authored text was often gathered from web scraping sites like convolutional neural network (CNN) or restaurant reviews [2], Kaggle competitions [5], or from existing open-source datasets such as the "essays with instructions" dataset available on Hugging Face or the dataset provided by Poetry-Foundation.org [8].

While most AI-generated text detectors are not better than a coin-flip in practice (and some are much worse), there are legitimate reasons someone might use an LLM that doesn't require the proverbial slap on the wrist of AI-detection. One could even argue that AI-assisted writing and translation lowers the barriers between people of different backgrounds and cultures to converse and share ideas. In addition to the text, features that could provide context to the text should be used to determine if messaging online is authentic or not.

Even if some uses of AI text generators are legitimate, we cannot ignore that AI-powered text generation is creating massive amounts of online data. In a 2023 article, Shumailov et al. coined the term "model collapse" [9]. Model collapse is a degenerative process that results in a model that no longer

represents the distribution of the underlying data. The probabilities of the AI-generated training data are compounded to create smaller probabilities for low-occurrence events and much higher probabilities for events that occur frequently. The authors call this the “curse of recursion” where much of the text found online could someday be generated by LLMs and thereby cause model collapse for future LLMs without a way to reliably detect authentic discourse from content that is model-generated. Knowing if the text was written by a human or not is imperative to its inclusion in LLM training data sets. LLMs can also be used for large-scale disenfranchisement of marginalized groups and exploiting systems and individuals for financial gain, among the many threats posed by public access. The ability to detect inauthentic discourse is essential to the preservation of democracy. Academic fraud stressing the publication review process, fake resumes, and cover letters are already impacting the culture as well as overall reduced trust in text communications, whereby users assume opposing viewpoints are inauthentic rather than acknowledge the existence of a variety of viewpoints, further dehumanizing and polarizing societies. Distrust is fostered by open-source data collected that may be impacted by model-poisoning attacks where enough text data of a certain type is produced to impact the samples and therefore the distribution of model training data. Generated text can be used in mass online influence campaigns that can be political (propaganda), social (disinformation), as well as commercial (fake reviews). Phishing and spam can also fit into this category since AI-generated text content in this context is centered around targeting and scalability. Translation models can be used to distribute this information across language barriers and bots can be used to ensure it reaches the maximum audience while social algorithms can be exploited to amplify the message. The platforms where this type of generated text is most prevalent should use reliable and fair methods to detect and remove inauthentic content before it can cause harm. Even given all this, platforms need to consider the harm of mass-censoring online speech AI-generated text detection methods [1].

To further investigate the potential of AI text detection, this research makes the following key contributions.

- 1) Introduce a balanced dataset of 10 000 records comprising both human- and AI-generated text.
- 2) Conduct a comprehensive analysis using various vectorization techniques and machine learning models to achieve high accuracy in distinguishing between human and AI-generated text.
- 3) Deploy our TF-IDF RF model as a user-friendly web application using the Streamlit framework, allowing users without coding expertise to interact with the model. The code is openly shared on GitHub for further collaboration and improvement.

II. METHODOLOGY

In this section, we outline the methodology implemented in our research. Fig. 1 provides a visual representation of this process, beginning with data sourcing that includes both human-generated and machine-generated texts. It then shows

the data cleansing process, followed by feature extraction. The final section of the figure outlines the modeling phases, where different machine learning algorithms were applied to the feature set for the development of the text classifier.

A. Data Collection and Preprocessing

To create our AI-generated text detection model, we first explored several means to acquire machine generated text and human generated text. Prerequisites included installing a virtual environment with the necessary packages as well as setting up the Kaggle command line interface (CLI) tool for downloading the human-generated text articles. We began with data acquisition and exploration, then model experimentation, research, and development, followed by results, validation, and explainability. A successful model is only as good as its data. It is important to have both human and machine generated text from diverse sources. We iteratively generated machine text by feeding various inputs to a Hugging Face DistilGPT2 model using a pipeline from the transformer package. During the development phase, we realized many of the outputs were similar unless the inputs were also very unique. Meanwhile, each execution of the DistilGPT2 pipeline used a single input and produced a single output. As we wanted thousands of diverse outputs, it was important to automate an iterative process using unique inputs. Thus, to minimize duplication, we used a unique input per output. DistilGPT2 is an English-language model pre-trained with the supervision of 124 million parameter version of GPT-2, an OpenAI product. DistilGPT2 has 82 million parameters and was developed using knowledge distillation, designed to be a faster, lighter version of GPT-2 [10]. On average, DistilGPT2 is two times faster than GPT-2. The distillation transformer has also been applied to various flavors of BERT such as RoBERTa to create DistilRoBERTa, German BERT to create German DistilBERT, and Multilingual BERT to create DistilMBERT which supports 104 languages [11].

Although GPT-4 was released in March 2023, our study utilized DistilGPT-2, chosen for its availability as a free API on Hugging Face. Considering the advancements in AI, future research could investigate the integration of more recent models such as GPT-4 to potentially enhance performance. In our current setup, producing a single record took a few seconds, as each input required a separate query to the API over the Internet connection. In future iterations of this research, we aim to improve the efficiency of data collection, possibly by enabling the retrieval of multiple records per API request.

The DistilGPT2 transformer takes any text prompt on which to concatenate more words using machine learning. The transformer also accepts parameters such as `max_length`, `pad_token_id`, and `num_return_sequences`. We attempted to extract the longest length by iterating through a high volume of a randomized selection of prompts. Using the highest value discovered to be possible (`max_length = 512`), a `pad_token_id` of 50256 (for open-ended text generation) and two `num_return_sequences`, we selected the longer of the two produced output texts.

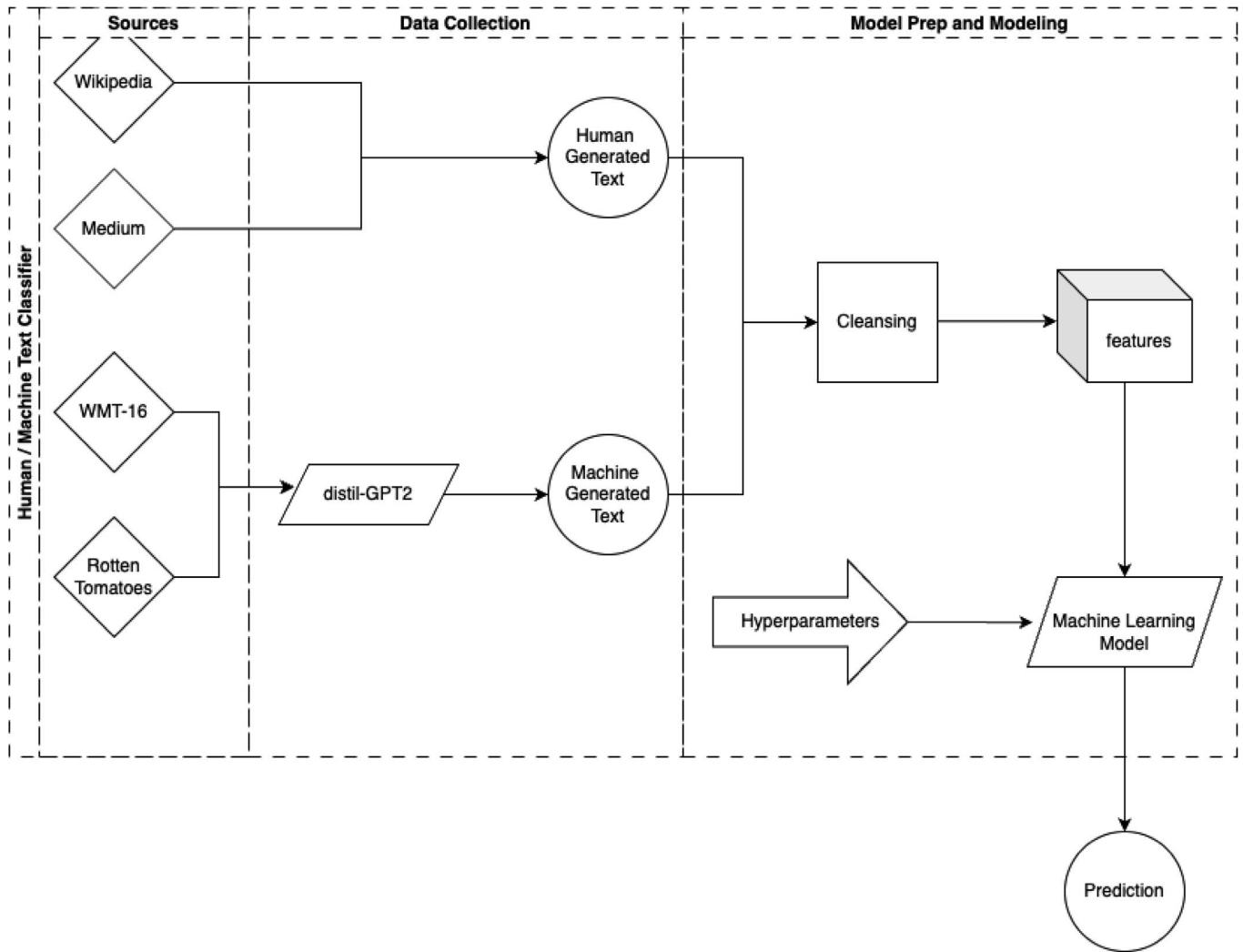


Fig. 1. Overview of the methodological process.

Two key datasets were used as inputs for the machine generated text model. This includes WMT16, a dataset typically used for translation. This dataset was desirable for this use case because it is a collection of key–value pairs, where the key is an English excerpt, and the value is a translated equivalent. WMT16 contains several languages. We extracted only the English excerpts, resulting in a long, diverse list of short English text blobs. The second dataset is English Rotten Tomatoes movie reviews, which are of a similar length as the WMT16 excerpts. Both datasets were loaded using the Hugging Face datasets package.

With a resulting 4665 records with Rotten Tomato inputs and 4533 machine-generated records with WMT16 inputs, the final pandas dataframe is modified to include four columns: one for the machine-generated text, one for the label (1 for machine), the input source (Rotten Tomato or WMT16), and the text word count. The resulting 9266 records with an average word count of 214 are saved as a csv.

Collecting human-generated text was much simpler using the Kaggle CLI tool for two different sources. First, over 185 000 Medium articles were downloaded with an average word count

of 834. Second, 19 990 Wikipedia articles were downloaded using the `load_dataset` module of the `datasets` package by Hugging Face. Wikipedia had an average word count of 3236. All of the Medium and Wikipedia articles were labeled as a 0 for human and saved as a csv. The resulting 200 000 or so human-generated records needed cleaning. To begin, the word counts were significantly higher than the machine generated text. To prevent word count bias causing the machine learning algorithms to predict human versus machine based on article length alone, we cleaned up the data among word counts.

In Fig. 2(a) and 2(b), the machine generated sources Rotten Tomato and WMT16 are labeled with a dark blue, with light blue representing human generated sources Medium and Wikipedia. On the left in Fig. 1(a), each point represents an article. The y-axis illustrates word counts. It is evident in Fig. 2(a) that the human generated sources have many more articles with larger word counts than the machine generated sources. Fig. 2(b) represents the dataset after cleansing both the human and machine generated sources of any article with a word count exceeding 450 or under 50, keeping 5000 records of each type, without duplication. After processing, the average machine

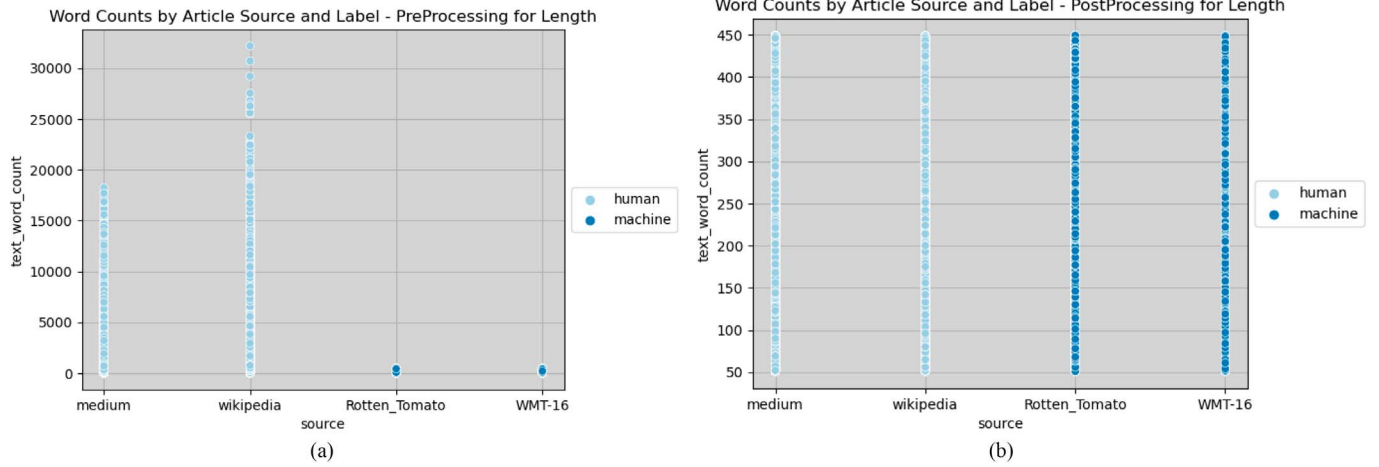


Fig. 2. Comparison of word counts before and after processing. (a) Word counts preprocessing. (b) Word counts postprocessing.

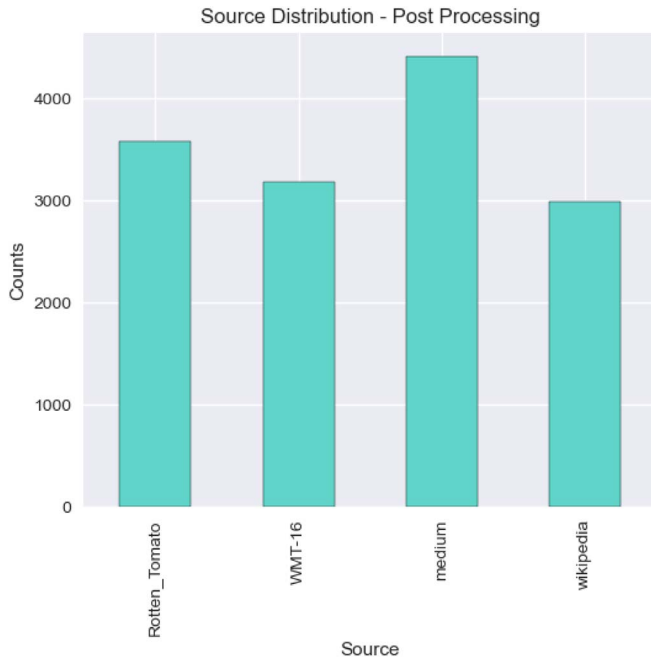


Fig. 3. Data labels by source.

generated word count is 276 and average human generated word count is 256.

We also visualized the dataset by label (machine or human, 1 and 0, respectively) and by source. This is shown in Fig. 3. This balanced distribution among source and word counts is much better for machine learning consumption, albeit the record count decreases to 10 000. In future work, we may increase the dataset size. However, this also permitted much faster processing times. Preprocessing steps also include data cleansing for punctuation, non-ascii characters, empty spaces, tabs, and end-of-the-line characters. Exploratory data analysis also included part-of-speech tagging for verbs, adjectives, adverbs, and nouns. The resulting vocab is 82 292 unique terms. This

vocab is used to control the size of the CountVectorizer and TfidfVectorizer matrix.

In our commitment to transparency and reproducibility, the dataset utilized in this study is publicly available for review and further research at <https://github.com/rsickle1/human-v-ai>.

B. Model Experimentation, Research, and Development

To optimize the balance between processing time and accuracy scores, several vectorization techniques were explored. These include bag of words (BoW), TF-IDF, BERT, and neural network embeddings with the keras package. BoW refers to a simple word frequency count. TF-IDF considers the term position in the document. BERT is calculated using the transformer mechanism and is able to represent text with full context.

The nature of word vectors is that they are highly dimensional to represent the text in multiple ways. To visualize highly dimensional data, dimensionality reduction techniques can be applied for representing two or three dimensions which can be more easily represented on a graph. We applied the UMAP algorithm to view BoW vectors by label and then by source in Fig. 4(a) and 4(b), respectively.

As the below representations are essentially clusters of the 2-D data, we would ideally see integrated colors. This would mean that the data does not follow any obvious pattern; the data is random. We don't see this in the BoW graphs, however. We have better success in the TF-IDF two-dimensional graphs in Fig. 5(a) and 5(b). although it still appears as if the source of the dataset tends to lean one way or another. We see this in Fig. 5(b) in the heavy cluster of red representing the Rotten Tomato dataset, Wikipedia in purple tends to lean lower left, and medium articles in green and WMT16 in orange are somewhere in the middle. The BERT embeddings in Fig. 6(a) and 6(b) appear much more spread out than the BoW and TF-IDF vectors, however, it still seems as if TF-IDF manages to integrate best of the three representations.

The BoW vectors were calculated using the CountVectorizer module from the sklearn package [12] using an ngram range of (1,2), a list of English stop words provided by the

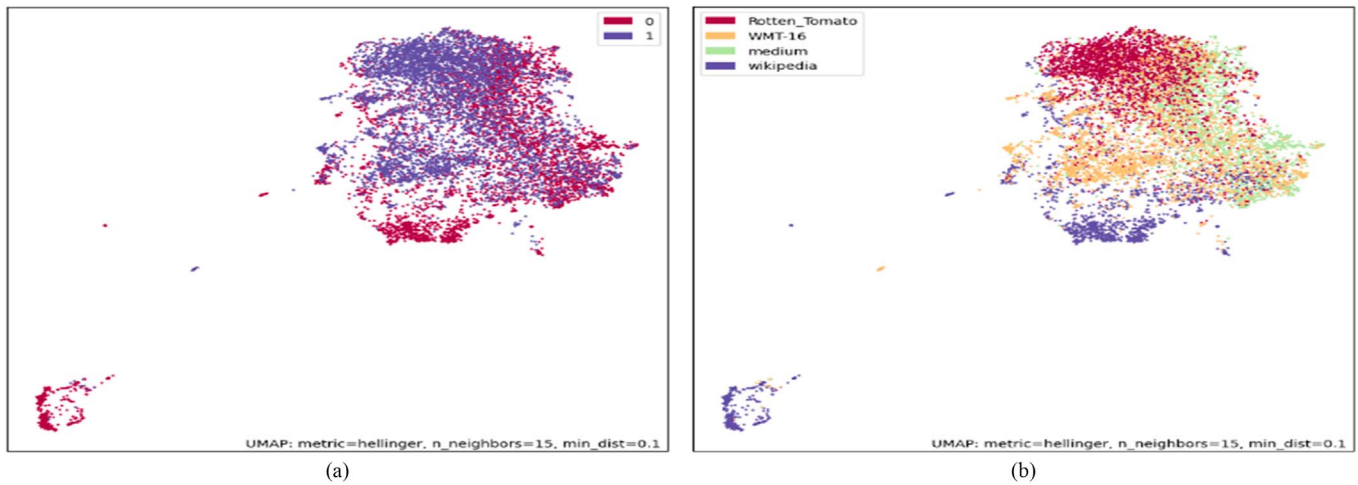


Fig. 4. UMAP visualizations of text data vectorized by BoW, categorized by label and source. (a) Distribution by label—depicting the clustering of human-generated. (b) Distribution by source—showing the separation of texts from Rotten Tomato, WMT-16, medium, and Wikipedia.

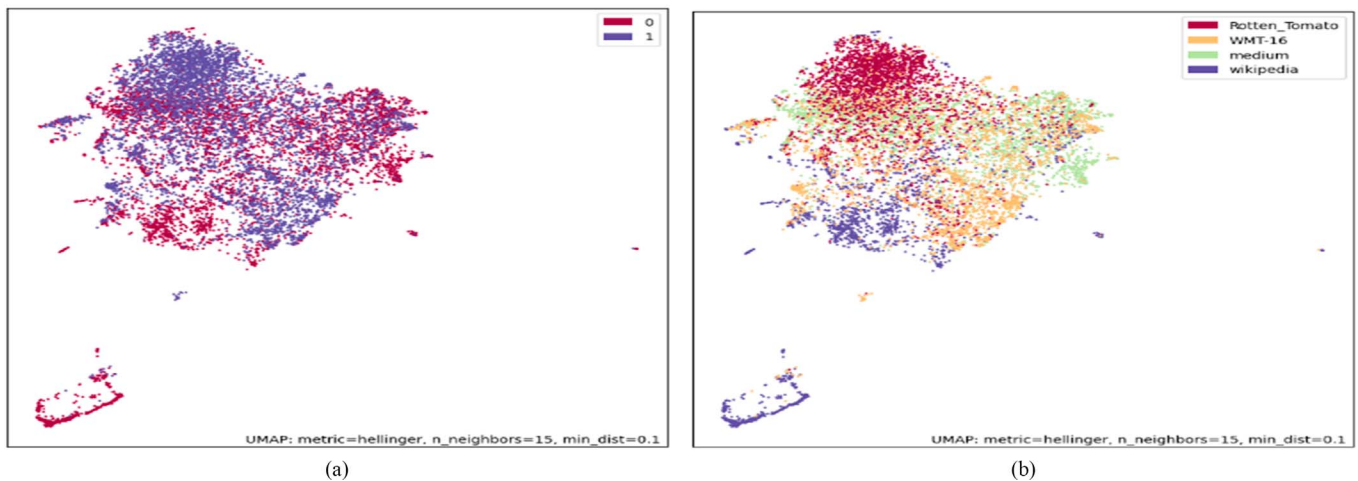


Fig. 5. UMAP visualizations of text data vectorized by TF-IDF, categorized by label and source. (a) Distribution by label—depicting the clustering of human-generated (0) and machine-generated (1) text. (b) Distribution by source—showing the separation of texts from Rotten Tomato, WMT-16, medium, and Wikipedia.

nlTK package, and a provided vocabulary list of about 82 292 terms which was derived during the cleansing preprocessing steps. Other parameters were explored but ultimately abandoned for being too memory intensive either to the point of crashing the local machine or without adding value to the model.

The TF-IDF vectors were calculated using the same n-gram range, the same stop words, but without the vocabulary list and instead controlling size using the `min_df` parameter with a value of 10. This parameter removes terms that appear too infrequently. Using a value of 10 ensures that any term used in the TF-IDF matrix has appeared in at least 10 documents (or records).

The BERT embeddings were calculated using the SentenceTransformer package, where `msmarco-distilbert-dot-v5`, `multi-qa-distilbert-cos-v1`, and `all-distilroberta-v1` were explored having a max sequence length of 512 and using 768 dimensions (each word is represented as a vector with a

length of 768). As another incentive to trim down the dataset, should the text exceed 450-word counts, this package would truncate any term count greater than 512. BERT embeddings are computationally expensive to calculate and took over an hour to compute on a Google Colab machine. Because of this, results were serialized using the pickle package and imported for future use, only being recalculated if the underlying dataset for some reason changed (such as new cleansing techniques).

From here, we feed each vectorization strategy to different models. These include logistic regression, RF, and XGBoost. Logistic regression [13] uses the sigmoid function to predict the probability of an outcome. In this case categorizing text as either machine-generated or human-generated. RF [14] employs a series of decision trees to produce an outcome. XGBoost, or extreme gradient boosting [15] creates an ensemble of weak decision trees sequentially. Each tree in XGBoost corrects the errors of its predecessors, enhancing the overall predictive accuracy.

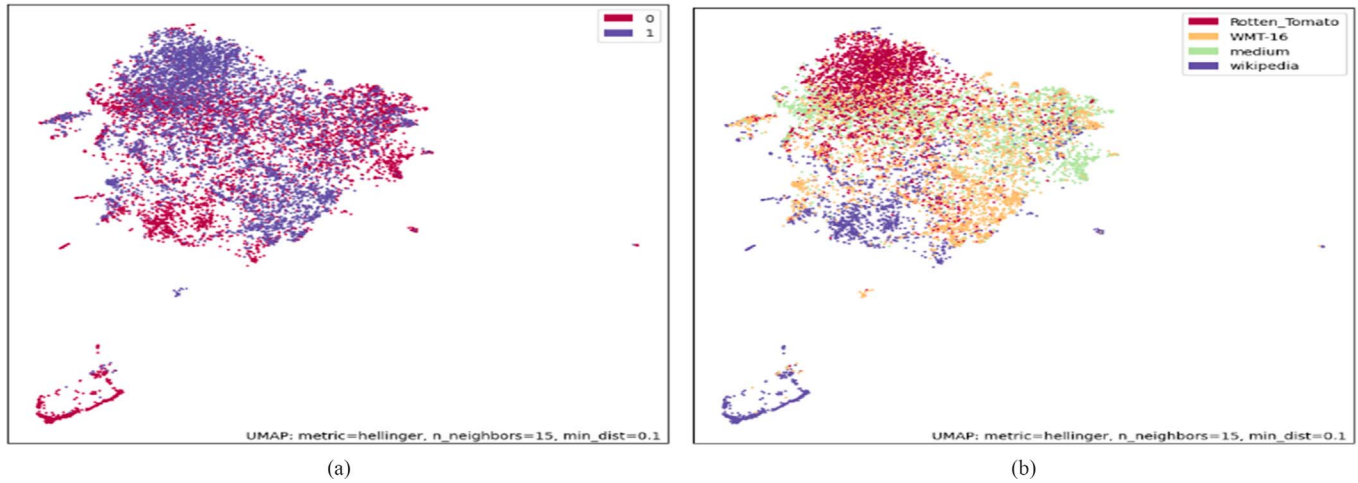


Fig. 6. UMAP visualizations of text data vectorized by BERT, categorized by label and source. (a) Distribution by label—depicting the clustering of human-generated (0) and machine-generated (1) text. (b) Distribution by source—showing the separation of texts from Rotten Tomato, WMT-16, medium, and Wikipedia.

TABLE I
CLASSIFICATION PERFORMANCE OF DIFFERENT MODELS ON
BoW REPRESENTATION

Classifier Type	Accuracy	Precision	Recall	F1 Score
Logistic regression	0.90	0.90	0.90	0.90
Random forest	0.87	0.88	0.87	0.87
XGBoost	0.91	0.91	0.91	0.91

TABLE II
CLASSIFICATION PERFORMANCE OF DIFFERENT MODELS ON
TF-IDF REPRESENTATION

Classifier Type	Accuracy	Precision	Recall	F1 Score
Logistic regression	0.90	0.90	0.90	0.90
Random forest	0.87	0.88	0.87	0.87
XGBoost	0.91	0.91	0.91	0.91

III. RESULTS AND DISCUSSION

TABLE III
CLASSIFICATION PERFORMANCE OF DIFFERENT MODELS ON
BERT REPRESENTATION

Classifier Type	Accuracy	Precision	Recall	F1 Score
Logistic	0.90	0.90	0.90	0.90
Random forest	0.81	0.81	0.81	0.81
XGBoost	0.83	0.83	0.83	0.83

A. Evaluation of Machine Learning Algorithms

In the first part of our study, we evaluated the performance of three traditional machine learning algorithms - logistic regression, RF, and XGBoost - across three different text vector representations: BoW [16], term frequency-inverse document frequency (TF-IDF) [17], and bidirectional encoder representations from transformers (BERT) [18]. This comprehensive analysis aims to determine how each algorithm performs with different representations and to identify the most effective combination for text classification tasks. Different parameters were explored for these models. Initially, we experimented with different cross-validation (CV) values [19]. Interestingly, a CV value of three yielded results comparable to those obtained with higher CV values, but with reduced computational time. Thus, we continued using three CV with the remaining models.

Table I presents the results for the BoW vector representation. The XGBoost classifier outperforms the others, delivering uniformly high scores of 0.91 across accuracy, precision, recall, and F1 metrics. This impressive performance underscores XGBoost's ability to effectively navigate the complexities of the BoW feature space. Its robust ensemble approach adeptly manages bias and variance within the dataset. Following closely is logistic regression, which demonstrates its effectiveness with balanced performance metrics, all approximately around the 0.90 mark.

The results for the TF-IDF representation are presented in Table II. Both the RF and XGBoost classifiers achieved identical scores across all metrics—accuracy, precision, recall, and F1 score—with each scoring 0.91. This outcome underscores the effectiveness of these algorithms in utilizing the TF-IDF representation for classification tasks. Logistic Regression, while marginally lower, still demonstrated commendable performance, maintaining scores of 0.90 across all metrics.

The evaluation of the BERT representation in Table III revealed a slightly divergent trend. While Logistic Regression sustained its performance, achieving a 0.90 score across all metrics, the RF and XGBoost classifiers experienced a diminution in performance, with scores descending to 0.81 and 0.83, respectively. This decrement may be attributed to the sophisticated nature of BERT embeddings, which could necessitate more complex model architectures or fine-tuning to fully capitalize on the contextual cues within the data. Logistic regression's consistency across all three representations substantiates its versatility and underlines its capability to deliver balanced

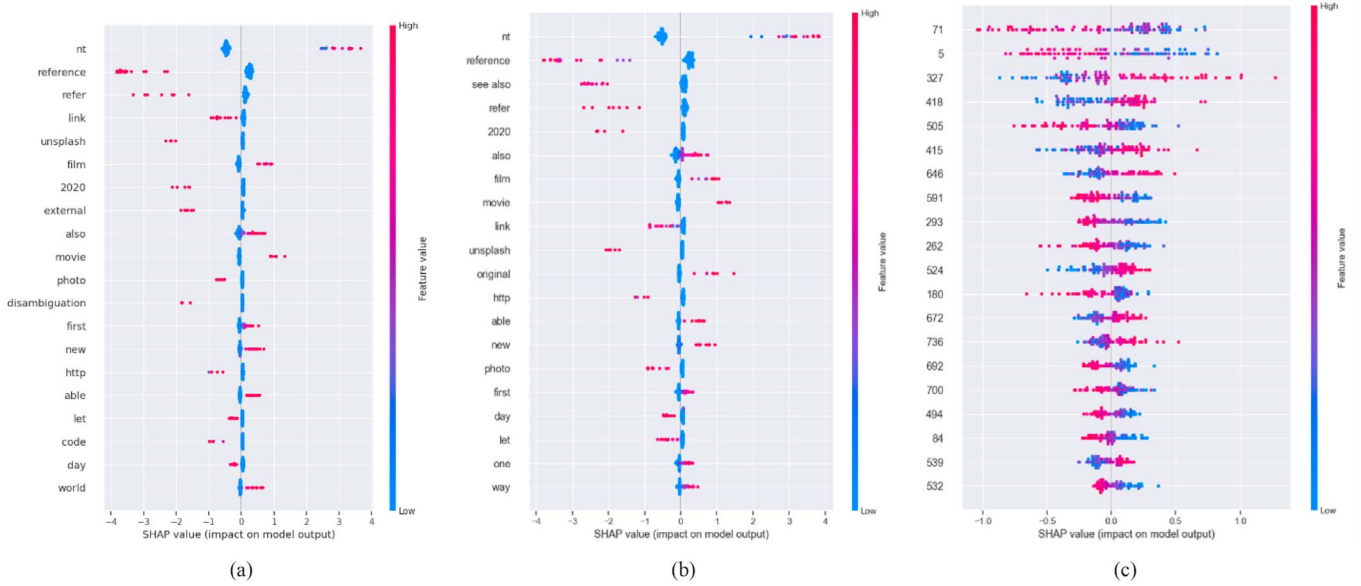


Fig. 7. SHAP summary plots for: (a) BoW; (b) TF-IDF; and (c) BERT vectorization strategies.

classification outcomes regardless of the vectorization technique employed. On the other hand, the decline in performance of RF and XGBoost when transitioning to BERT embeddings merits further investigation. This could potentially suggest that while these algorithms excel with traditional vector representations, they may not be as adept at harnessing the full potential of contextual embeddings without significant adjustments to their configuration.

To further our understanding of the decision-making processes underlying our machine learning models, we employed Shapley additive explanations (SHAP) analysis [20]. We used the XGBoost model to most easily integrate with the SHAP package's Tree Explainer to produce the below summary plots per vectorization strategy. The X data is the count matrix or TF-IDF matrix or in the case of BERT, the BERT embeddings. A dataframe of the X training data is sampled randomly for 100 records, upon which SHAP values are calculated. This can be an expensive process, hence the random sampling of 100. The summary plot takes the parameters SHAP values and the 100 randomly selected X training records. The results are given in Fig. 7. The purpose of these plots is to demonstrate based on SHAP values what is likely causing the positive, negative, or neutral impact on the model output. SHAP values range from -4 , to 0 , to 4 . A value of 0 for some feature means that in that observation, the value of that feature is ignored by the model; the model is neutral to this feature. SHAP values of 4 are strongly positive; the value of that feature in the current example increases the model output by 4 .

For example, in the BoW and TF-IDF models, the term *nt* is often associated with a predicament in which the model makes a positive prediction (machine-generated text). Meanwhile, the term *reference* often causes the model to make a negative prediction (human-generated text). The term *link* is consistently more valuable to the model than the term *let*.

However, for the BERT SHAP model, we can see much more activity per term, but the terms are ambiguous. Indeed, the most important takeaway from this figure is that BERT embeddings cannot be explained simply. In regulatory settings with requests for right to explanation, transformer-based AI is much more complicated to provide straightforward explanation.

B. Exploration of Deep Learning Techniques

The latter phase of our study concentrated on evaluating deep learning strategies for AI-generated text detection, utilizing three distinct neural network architectures: LSTM [21], CNN [22], and a hybrid CNN-LSTM model [23]. These models were implemented using the Keras library [24]. Our dataset underwent tokenization via 'Keras' Tokenizer, subsequently forming the basis for model input. Each model was constructed with an Embedding layer designed to handle a vocabulary size of 10 000 words, using an embedding dimension of 16 to represent these words densely, and processing sequences with a maximum length of 500 tokens. This setup was consistent across the LSTM, CNN, and CNN-LSTM models, ensuring a uniform foundation for feature representation. The LSTM network was structured sequentially, incorporating an embedding layer, an LSTM layer with 32 units, and a Dense layer with a sigmoid activation function for binary classification. Our CNN architecture replaced the LSTM layer with a Conv1D layer, also with 32 units, and included a flatten layer to process the spatial information. The CNN-LSTM combined the two approaches, utilizing both Conv1D and LSTM layers with 32 units each, aiming to exploit the advantages of convolutional feature extraction and sequence processing. To prevent overfitting and optimize computational efficiency, early stopping was employed during the training phase. The effectiveness of this approach is illustrated in Fig. 8 which provides the training and validation accuracy and loss curves for each of the deep learning models.

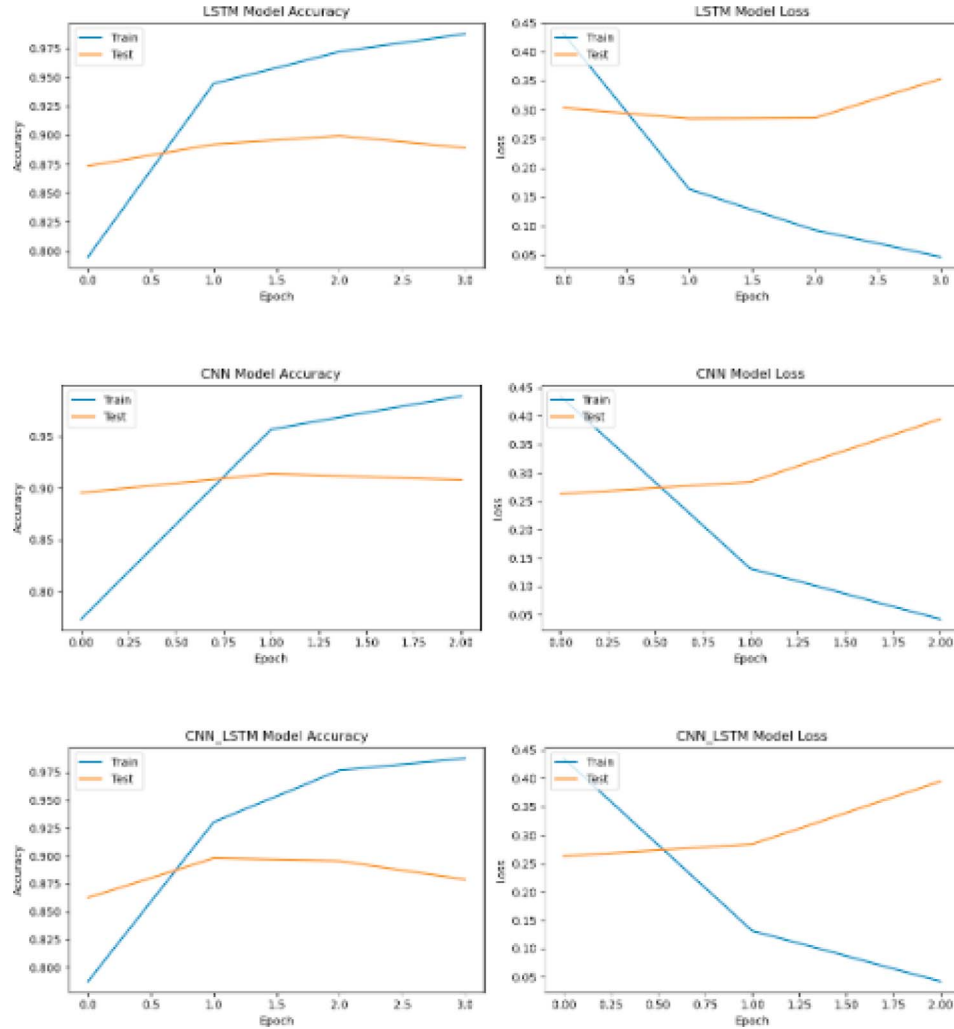


Fig. 8. Training and validation accuracy and loss for LSTM, CNN, and CNN-LSTM models.

TABLE IV
HYPERPARAMETERS FOR DEEP LEARNING MODELS

Model	Max Features	Maxlen	Embedding Dim	Units	Training Time (s)
LSTM	10000	500	16	LSTM: 32	107.31
CNN	10000	500	16	Conv: 32	9.21
CNN-LSTM	10000	500	16	Conv: 32, LSTM: 32	127.19

These curves are pivotal for understanding the models' learning behavior over epochs and for identifying potential issues such as overfitting or underfitting. We summarize the hyperparameters and the final results obtained at the last epoch before early stopping in Tables IV and V. The LSTM model demonstrated an accuracy of 98.75% on the training data and 88.90% on the validation data, with a notable loss of 0.3532 on the validation set after four epochs. The CNN model, on the other hand, achieved a slightly higher accuracy of 98.90% on the training data and 90.80% on the validation data, but with a higher validation loss of 0.3950 after three epochs. The CNN-LSTM model's performance was slightly lower, with an accuracy of 98.76% on the training data and 87.90% on the validation data, resulting in a validation loss of 0.3986 after four epochs. Notably, We found that the CNN model trained fastest and provided the best

results, making it a primary candidate for further research and optimization.

C. Deployment to a Web Application

The underlying mechanism of the application involves deserializing a pretrained sklearn pipeline object, enabling the input text—such as the example shown in Fig. 9, captured from an AI-related Wikipedia page—to be processed and classified. The application translates the binary classification result into a user-friendly output displayed on the UI.

In an effort to bridge the gap between technical research and practical utility, We deployed the TF-IDF random forest model from the experiments above to a web application with a user-friendly user interface (UI). We created this application

TABLE V
FINAL TRAINING AND VALIDATION RESULTS FOR DEEP LEARNING MODELS

Model	Train Loss	Train Accuracy	Val Loss	Val Accuracy	Epochs
LSTM	0.0467	98.75%	0.3532	88.90%	4
CNN	0.0423	98.90%	0.3950	90.80%	3
CNN-LSTM	0.0447	98.76%	0.3986	87.90%	4

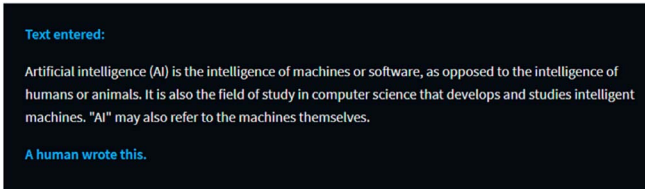


Fig. 9. Example output displayed by the web application, using text from the AI Wikipedia page as input.

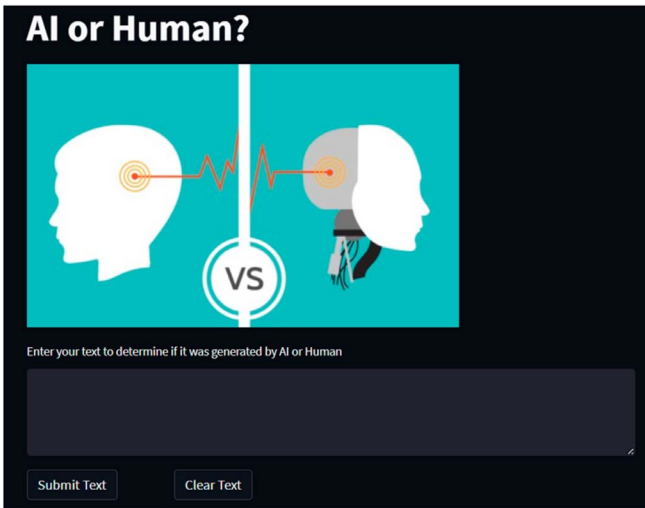


Fig. 10. Screenshot of the Streamlit web application user interface.

using the Streamlit framework (<https://streamlit.io/>) which provides components to allow the developer to create a frontend application using pure Python. This removes the barriers to most machine learning and data science professionals that would prevent them from providing their models to users who don't write Python code. This is also a way to allow users outside of the coding community to test models and provide feedback to the technical resources responsible for training the models. The application could also be deployed to a web server to make it publicly available. The GitHub repository for this application can be accessed here: <https://github.com/rsickle1/human-v-ai>.

The user interface, depicted in Fig. 10, showcases the simplicity and intuitiveness of the application, allowing users to input text and receive model predictions with ease.

This streamlined interface ensures that the model is not only accessible to those outside the technical domain but also provides a platform for extensive user feedback. Additionally, the application is structured to be deployed on a web server, thereby enhancing its accessibility to a wider audience.

The flexibility of the application's architecture is such that any serialized model, equipped with the requisite preprocessing steps and capable of binary inference, can be integrated into the system. This facilitates a collaborative environment where machine learning experts can deploy their models and garner feedback from a diverse set of users, including subject matter experts. Such a practice is instrumental in refining and advancing AI-detection systems, making them more robust and reliable.

D. Limitations of the Study

This research, while comprehensive in its approach and methodologies, encounters several significant limitations that warrant consideration.

A primary limitation lies in the inherent "black box" nature of the deep learning models utilized in the study. These models, particularly the complex architectures such as CNNs and LSTMs, often operate without transparent interpretability. Despite their proven effectiveness in text classification tasks, the lack of clarity in how these models process inputs and make predictions poses a considerable challenge. This aspect is particularly crucial in fields where the need for explainability and accountability is paramount. The inability to fully understand and explain the specific internal mechanisms and reasoning of these models restricts their application in scenarios that demand high levels of transparency and interpretability.

Another constraint of this study is related to the dataset size and quality. The dataset employed for the initial exploration and training of models, while sufficient for preliminary investigations, presents potential for enhancement in terms of volume and diversity. Improving the dataset, as discussed in the Future Work section of this study, could lead to significant advancements in the model's performance and its generalizability. Expanding the dataset size and enhancing its quality are crucial for developing more robust and reliable AI-detection systems. Addressing these limitations will not only improve the efficacy of the models but also extend their practical applicability in diverse real-world contexts.

IV. CONCLUSION AND FUTURE WORK

In this study, our primary contribution was the creation of a novel dataset of 10 000 records, comprising both human and machine-generated text. We leveraged this dataset to develop and test several models, including conventional machine learning algorithms and deep learning algorithms. We successfully deployed one of these models, capable of predicting whether a body of text is generated by a machine or a human. This

contribution is crucial given the increasing prevalence of machine-generated content in online narratives, which may not always be fact-checked or scientifically accurate. We did not compare our results with existing studies, as we introduced and utilized a new dataset to apply various models. Our analysis revealed that conventional machine learning models, such as XGBoost, logistic regression, and RF, performed robustly with traditional vectorization techniques such as BoW and TF-IDF. This performance is likely due to the straightforward nature and manageable size of the dataset. In contrast, deep learning models showed a slightly lower performance. These results suggest that deep learning models require larger and more diverse datasets to fully leverage their potential for capturing contextual nuances.

In the future, we will utilize the paid GPT-4 service instead of the open-source DistilGPT-2 LLM, potentially providing more sophisticated and accurate outputs. Expanding and refining the dataset could also yield improvements. For instance, incorporating more diverse datasets could address issues related to English as a second language writing styles, thereby achieving more authentic linguistic diversity. Increasing the dataset size could further enhance model performance, providing more comprehensive training data for both conventional and deep learning models. Additionally, future research could streamline the current manual process of crafting prompts for the AI model by automating prompt creation. This automation could significantly improve the efficiency and scalability of the data generation process. Furthermore, using Wikipedia articles published before ChatGPT's release in November 2022 would ensure that all human-labeled data are genuinely human-generated. Finally, exploring faster data collection methods, such as obtaining multiple records in bulk per API request, could expedite the process and improve the overall efficiency of data acquisition.

REFERENCES

- [1] E. Crothers, N. Japkowicz, and H. L. Viktor, "Machine-generated text: A comprehensive survey of threat models and detection methods," *IEEE Access*, vol. 11, pp. 70977–71002, 2023.
- [2] N. Islam, D. Sutradhar, H. Noor, J. T. Raya, M. T. Maisha, and D. M. Farid, "Distinguishing human generated text from ChatGPT generated text using machine learning," 2023, *arXiv:2306.01761*.
- [3] S. Mitrović, D. Andreoletti, and O. Ayoub, "Chatgpt or human? detect and explain. explaining decisions of machine learning model for detecting short ChatGPT-generated text," 2023, *arXiv:2301.13852*.
- [4] R. Zellers et al., "Defending against neural fake news," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [5] V. S. Sadasivan, A. Kumar, S. Balasubramanian, W. Wang, and S. Feizi, "Can AI-generated text be reliably detected?" 2023, *arXiv:2303.11156*.
- [6] C. Chaka, "Detecting AI content in responses generated by ChatGPT, youChat, and chatsonic: The case of five AI content detection tools," *J. Appl. Learn. Teach.*, vol. 6, no. 2, 2023.
- [7] W. Liang, M. Yuksekgonul, Y. Mao, E. Wu, and J. Zou, "GPT detectors are biased against non-native English writers," 2023, *arXiv:2304.02819*.
- [8] K. Hayawi, S. Shahriar, and S. S. Mathew, "The imitation game: Detecting human and ai-generated texts in the era of large language models," 2023, *arXiv:2307.12166*.
- [9] I. Shumailov, Z. Shumaylov, Y. Zhao, Y. Gal, N. Papernot, and R. Anderson, "The curse of recursion: Training on generated data makes models forget," 2023, *arxiv:2305.17493*.
- [10] H. Face, "Distilgpt2." Accessed: Jan. 2, 2024. [Online]. Available: <https://huggingface.co/distilgpt2>
- [11] H. Face, "Transformers." Accessed: Jan. 5, 2024. [Online]. Available: https://github.com/huggingface/transformers/tree/main/examples/research_projects/distillation
- [12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [13] D. R. Cox, "The regression analysis of binary sequences," *J. Roy. Stat. Soc. Ser. B: Stat. Methodol.*, vol. 20, no. 2, pp. 215–232, 1958.
- [14] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, 2001.
- [15] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.
- [16] A. Al Bataineh and D. Kaur, "Immunocomputing-based approach for optimizing the topologies of LSTM networks," *IEEE Access*, vol. 9, pp. 78993–79004, 2021.
- [17] A. Al Bataineh and S. Manacek, "MLP-PSO hybrid algorithm for heart disease prediction," *J. Personalized Med.*, vol. 12, no. 8, p. 1208, 2022.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [19] A. Al Bataineh, D. Kaur, and S. M. J. Jalali, "Multi-layer perceptron training optimization using nature inspired computing," *IEEE Access*, vol. 10, pp. 36963–36977, 2022.
- [20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [21] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [22] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [23] J. Donahue et al., "Long-term recurrent convolutional networks for visual recognition and description," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 2625–2634.
- [24] F. Chollet et al. (2015). "Keras." Accessed: Jan. 13, 2024. [Online]. Available: <https://github.com/keras-team/keras>



Ali Al Bataineh received the Ph.D. degree in electrical engineering with a focus on artificial intelligence.

He is the Director of the Artificial Intelligence Center and an Assistant Professor of Electrical and Computer Engineering with Norwich University, Northfield, VT, USA. His research interests include AI-driven optimization, natural language processing, and machine learning applications in both academic and industrial settings. He has collaborated on multiple grants from the NSF, NIH, Department

of Education, and other organizations, advancing research in AI and related fields.



Rachel Sickler received the bachelor's degree in accounting, business, and computer science and the M.S. degree in data science from the Northwestern University.

Currently, she is a Senior Developer and a Machine Learning Engineer with Norwich University Applied Research Institutes, Northfield, VT, USA. Her experience includes software engineering, business analytics, natural language processing, predictive analytics, model observability and explainability, end-to-end data engineering, and machine learning engineering. She holds bachelor's degrees in Accounting, Business and Computer Science and a M.S. in Data Science from Northwestern University. Her research interests include social cybersecurity, criminal justice, and artificial intelligence ethics.



Kerry Kurcz received the master's degree in data science from the Northwestern University.

Currently, she is an Artificial Intelligence (AI) Solution Architect with the Northwestern University, specializing in AI research on embedding strategies. Passionate about delivering AI solutions that drive business impact, she bridges the gap between technical and business perspectives for fast, scalable implementation. She leads strategic AI initiatives and fosters effective cross-team communication to launch high-value AI products.



Kristen Pedersen received the Ph.D. degree.

Currently, she is the Vice President and the Chief Research Officer with Norwich University Applied Research Institutes, Northfield, VT, USA. She is responsible for NUARI's cyber operations and portfolio of national security contracts and research projects for Department of Homeland Security (DHS), Department of Defense (DOD), National Security Agency (NSA), and National Critical Infrastructure Organizations. Her research interests include influence/information operations and cyberpsychology.